C Programming Lab Guide

Part I — C String Fundamentals You Need First

1. What is a string in C?

A string is stored as an array of characters followed by a special character called the null character, `'\0'`.

```
char s[] = "APPLE";
char s[] = {'A','P','P','L','E'};
```

Memory concept:

```
A  P  P  L  E  \0
0  1  2  3  4   5   <- indices
```

Important: The visible characters are A, P, P, L, E. The `'\0'` is not normally printed, but it tells C where the string ends.

2. Reading a complete line including spaces

`scanf("%s", s)` stops reading at whitespace. Therefore it is not suitable when the input contains spaces.

```
char s[200];
fgets(s, sizeof(s), stdin);
```

`fgets()` can read spaces and stops when it reaches a newline, the requested limit, or EOF. When the input line contains a newline, `fgets()` usually stores that newline too.

```
s[strcspn(s, "\n")] = '\0';
```

The line above removes the newline character at the of input. It uses `strcspn()`, available in `<string.h>`, to find the first newline.

3. Finding string length: stop at '\0'

The basic idea behind `strlen()` is simply to count characters until `'\0'` is reached.

```
int len = 0;

while (s[len] != '\0') {
    len++;
}
```

For "APPLE", the loop counts A, P, P, L, E and stops when it reaches `'\0'`.

Related function: `strlen(s)` from `<string.h>` performs this operation for you.

4. Character case: uppercase, lowercase, or ignore case

Characters have numeric codes. C provides functions in `<ctype.h>` that make case handling easier.

```
#include <ctype.h>

char c = 'A';

c = tolower(c);   // becomes 'a'
c = toupper(c);   // becomes 'A'
```

```
if (tolower(a) == tolower(b)) {
    // same ignoring case
}
```

For a comparison that ignores case, convert both characters to the same case before comparing.

```
if (tolower(s[i]) == tolower(t[i])) {
    ...
}
```

Common mistake: `tolower()` and `toupper()` return an int. In normal ASCII-style character processing, assigning the result to char is fine, but the important idea is to use the converted value in the comparison.

5. Removing spaces and other unwanted characters

If a question says to ignore spaces and other characters, do not compare the original strings directly. Instead, scan the string and process only the characters you care about.

```
for (int i = 0; s[i] != '\0'; i++) {
    if (isalpha((unsigned char)s[i])) {
        // process this letter
    }
}
OR
for (int i = 0; s[i] != '\0'; i++) {
    if (isspace(s[i])) {
        continue;
        //skip this char
    }
}
```

`isalpha()` keeps letters and ignores spaces, digits, punctuation, etc. If the question wants letters and digits, use `isalnum()` instead. `isspace()` identifies whitespace, tab, newline,... characters.

Part II — Lab 4: Digits of an Integer

Question 6(b): Print digits from right-to-left

Question: Accept a positive integer n and print its decimal digits from right-to-left. Example: 2026 → 6, 2, 0, 2.
The key idea: `% 10` and `/ 10`
For an integer n, `n % 10` gives the last digit. Integer division `n / 10` removes the last digit.

```
n = 2026

n % 10 = 6
n / 10 = 202

202 % 10 = 2
202 / 10 = 20

20 % 10 = 0
20 / 10 = 2

2 % 10 = 2
2 / 10 = 0
```

So the loop is naturally:

```
while (n > 0) {
    digit = n % 10;
    printf("%d,", digit);
    n = n / 10;
}
```

Complete program:

```
#include <stdio.h>

int main(void) {
    int n, digit;

    printf("Enter␣a␣positive␣integer:␣");
    scanf("%d", &n);

    while (n > 0) {
        digit = n % 10;
        printf("%d", digit);

        n = n / 10;

        if (n > 0)
            printf(",␣");
    }

    printf("\n");
    return 0;
}
```

For 2026: output sequence = 6 → 2 → 0 → 2.

Question 6(c): Print digits from left-to-right without an array

Now the problem is harder because `% 10` gives digits from the right. We need the first digit first.
There are two useful approaches. For this lab, the easiest is to first find the largest power of 10.

```
2026

power = 1000

2026 / 1000 = 2     -> first digit
2026 % 1000 = 26

26 / 100 = 0        -> next digit
26 % 100 = 26

26 / 10 = 2
26 % 10 = 6
```

Algorithm:

1. Find the largest power of 10 that is <= n.
2. Take n / power to get the leftmost digit.
3. Remove that digit using n % power.
4. Divide power by 10 and repeat.

```
#include <stdio.h>

int main(void) {
    int n, temp, power = 1;

    printf("Enter␣a␣positive␣integer:␣");
    scanf("%d", &n);

    temp = n;

    while (temp >= 10) {
        temp = temp / 10;
        power = power * 10;
    }

    while (power > 0) {
        printf("%d", n / power);

        n = n % power;
        power = power / 10;

        if (power > 0)
            printf(",␣");
    }

    printf("\n");
    return 0;
}
```

Why no array?: We never store the digits. We use division and remainder to extract them directly.

Trailing zeros: an important edge case

A common trap is that an integer such as 1200 and the digit sequence "1200" are not the same thing as a number after arithmetic operations. When you reverse digits numerically, leading zeros in the reversed result disappear.
Example:

```
1200 -> right-to-left digits: 0, 0, 2, 1
numeric reverse: 21
```

If the question requires the zeros to be displayed, print each extracted digit immediately instead of storing the reversed number. If you are asked to store the reversed number, keep a separate count of trailing zeros before they disappear.

Storing the reversed number and tracking zeros

```
int n, rev = 0, zero_count = 0;

scanf("%d", &n);

while (n > 0) {
    int digit = n % 10;

    if (rev == 0 && digit == 0)
        zero_count++;

    rev = rev * 10 + digit;
    n = n / 10;
}

printf("Reversed␣number␣=␣%d\n", rev);
printf("Zeros␣at␣the␣beginning␣of␣the␣reversed␣representation␣=␣%d\n",
       zero_count);
```

Part III — Lab 4 Question 8: arctan(x) Using Taylor Series

1. Logic / Approach

We need to calculate:

$$\tan^{-1}(x) = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \cdots$$

We have to add the first 20 terms.
The pattern is:

```
Power:       1, 3, 5, 7, ...
Denominator: 1, 3, 5, 7, ...
Sign:        +, -, +, -, ...
```

For term number i:

```
power = 2*i + 1
denominator = 2*i + 1
```

The sign depends on whether i is even or odd:

```
i = 0 -> +
i = 1 -> -
i = 2 -> +
i = 3 -> -
```

So:

```
if (i % 2 == 0)
    sum += term;
else
    sum -= term;
```

The `pow()` function from `<math.h>` is used to calculate the power of x.

2. C Program

```
#include <stdio.h>
#include <math.h>

int main()
{
    double x, sum = 0.0;

    printf("Enter␣x␣(-1,␣1):␣");
    scanf("%lf", &x);

    for (int i = 0; i < 20; i++)
    {
        int power = 2 * i + 1;
        double term = pow(x, power) / power;

        if (i % 2 == 0)
            sum += term;
        else
            sum -= term;
    }

    printf("Taylor␣approximation␣=␣%.12f\n", sum);
    printf("Built-in␣atan(x)␣␣␣␣␣␣=␣%.12f\n", atan(x));

    return 0;
}
```

3. Code Explanation

Calculating the power:

```
int power = 2 * i + 1;
```

This generates:

```
i = 0 -> 1
i = 1 -> 3
i = 2 -> 5
i = 3 -> 7
```

Calculating each term:

```
double term = pow(x, power) / power;
```

For example, if i = 1:

```
power = 3
```

```
term = x^3 / 3
```

Alternating the sign:

```
if (i % 2 == 0)
    sum += term;
else
    sum -= term;
```

i % 2 gives the remainder after dividing by 2.

```
Even -> remainder 0 -> add
Odd  -> remainder 1 -> subtract
```

Comparing with atan():

```
atan(x)
```

returns the built-in value of $\tan^{-1}(x)$ from `<math.h>`.

Part IV — Lab 4 Optional: Left Rotate a String k Times

1. Logic / Approach

A left rotation by one position means:

```
APPLE
  |
PPLEA
```

The first character moves to the end, and every other character shifts one position to the left.
For example:

```
CHROME

1 rotation -> HROMEC
2 rotations -> ROMECH
3 rotations -> OMECHR
4 rotations -> MECHRO
```

For every rotation:
1. Store the first character temporarily.
2. Shift all remaining characters one position to the left.
3. Put the saved first character at the end.
4. Repeat this k times.

We do not need a second string.
Also:

```
k = k % len;
```

because rotating a string by its length brings it back to the original string.

2. C Program

```
#include <stdio.h>
#include <string.h>

int main()
{
    char s[200];
    int k;

    printf("Enter␣the␣string:␣");
    fgets(s, sizeof(s), stdin);

    s[strcspn(s, "\n")] = '\0';

    printf("Enter␣the␣number␣of␣left-rotations:␣");
    scanf("%d", &k);

    int len = strlen(s);

    if (len > 0)
    {
        k = k % len;

        for (int r = 0; r < k; r++)
        {
            char first = s[0];

            for (int i = 0; i < len - 1; i++)
            {
                s[i] = s[i + 1];
            }

            s[len - 1] = first;
        }
    }

    printf("The␣rotated␣string␣is:␣%s\n", s);

    return 0;
}
```

3. Code Explanation

Reading the string:

```
fgets(s, sizeof(s), stdin);
```

`fgets()` is used instead of `scanf("%s", s)` because it can read a complete line containing spaces.
Then:

```
s[strcspn(s, "\n")] = '\0';
```

removes the newline stored by `fgets()`.
Finding the length:

```
int len = strlen(s);
```

`strlen()` returns the number of characters before `'\0'`.
Avoiding unnecessary rotations:

```
k = k % len;
```

For example, if:

`APPLE`

length = 5
then:

`k = 7`

is equivalent to:

`k = 7 % 5 = 2`

So we only need 2 rotations.
Saving the first character:

```
char first = s[0];
```

For:

`APPLE`

we temporarily store:

`first = 'A'`

Shifting characters:

```
for (int i = 0; i < len - 1; i++)
{
    s[i] = s[i + 1];
}
```

This changes:

`APPLE`

to:

`PPLE?`

The last position is still available.
Putting the first character at the end:

```
s[len - 1] = first;
```

So:

`PPLE? -> PPLEA`

The outer loop repeats this process k times.

Part V — Lab 4 Optional: 15-Puzzle — One Move

The board is a 4×4 matrix. Use 0 to represent the blank. A move means swapping the blank with one neighboring cell.

The key idea: locate the blank

```
int blank_row, blank_col;

for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 4; j++) {
        if (board[i][j] == 0) {
            blank_row = i;
            blank_col = j;
        }
    }
}
```

Once the blank position is known, N/W/E/S tells us which neighboring cell should be swapped with it.

Direction logic

Remember: the direction describes where the blank moves.

```
N: blank swaps with board[row - 1][col]
S: blank swaps with board[row + 1][col]
W: blank swaps with board[row][col - 1]
E: blank swaps with board[row][col + 1]
```

Before swapping, check that the target position is inside the 4×4 board.

How to swap two variables

```
int temp;

temp = a;
a = b;
b = temp;
```

For example, if `board[r][c]` is 0 and `board[r][c+1]` is 12:

```
temp = board[r][c];
board[r][c] = board[r][c + 1];
board[r][c + 1] = temp;
```

Complete 15-puzzle program

```
#include <stdio.h>

void print_board(int board[4][4]) {
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (board[i][j] == 0)
                printf("␣␣␣");
            else
                printf("%2d␣", board[i][j]);
        }
        printf("\n");
    }
}

void move_blank(int board[4][4], char move) {
    int r = -1, c = -1;

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (board[i][j] == 0) {
                r = i;
                c = j;
            }
        }
    }

    int nr = r, nc = c;

    if (move == 'N') nr = r - 1;
    else if (move == 'S') nr = r + 1;
    else if (move == 'W') nc = c - 1;
    else if (move == 'E') nc = c + 1;
    else {
        printf("Use␣N,␣S,␣E,␣W,␣or␣X.\n");
        return;
    }

    if (nr < 0 || nr >= 4 || nc < 0 || nc >= 4) {
        printf("Invalid␣move.\n");
        return;
    }

    int temp = board[r][c];
    board[r][c] = board[nr][nc];
    board[nr][nc] = temp;
```

```
}

int main(void) {
    int board[4][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 0},
        {13, 14, 15, 12}
    };

    char move;

    print_board(board);

    while (1) {
        printf("Enter␣move␣(N/W/E/S,␣X␣to␣stop):␣");
        scanf("␣%c", &move);

        if (move == 'X')
            break;

        move_blank(board, move);
        print_board(board);
    }

    return 0;
}
```

Why `scanf(" \%c", \&move)`?: The leading space tells scanf to skip leftover whitespace/newlines before reading the next character.

Part VI — Lab 6: Palindrome with Spaces and Special Characters

The common requirement is: check whether a string is a palindrome after ignoring spaces, punctuation, and letter case. Example:

```
A man, a plan, a canal: Panama
-> amanaplanacanalpanama
-> palindrome
```

The important logic

Do not compare the original string directly. Use two indices: left starts at the beginning and right starts at the end.

5. Move left forward while the character is not relevant.
6. Move right backward while the character is not relevant.
7. Compare the two relevant characters after converting them to the same case.
8. If they differ, it is not a palindrome.
9. Otherwise move inward: left++, right–.

```
while (left < right) {
    while (left < right && !isalnum((unsigned char)s[left]))
        left++;

    while (left < right && !isalnum((unsigned char)s[right]))
        right--;

    if (tolower((unsigned char)s[left]) !=
        tolower((unsigned char)s[right])) {
        palindrome = 0;
        break;
    }

    left++;
    right--;
}
```

Complete palindrome program

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main(void) {
    char s[300];
    int left = 0, right;
    int palindrome = 1;

    printf("Enter␣a␣string:␣");
    fgets(s, sizeof(s), stdin);
    s[strcspn(s, "\n")] = '\0';

    right = strlen(s) - 1;

    while (left < right) {
        while (left < right &&
               !isalnum((unsigned char)s[left]))
            left++;
```

```
        while (left < right &&
               !isalnum((unsigned char)s[right]))
            right--;

        if (tolower((unsigned char)s[left]) !=
            tolower((unsigned char)s[right])) {
            palindrome = 0;
            break;
        }

        left++;
        right--;
    }

    if (palindrome)
        printf("Palindrome\n");
    else
        printf("Not␣a␣palindrome\n");

    return 0;
}
```

If the question says to ignore only spaces but not punctuation, change the filtering condition accordingly. Always read the exact wording before choosing `isalpha()`, `isalnum()`, or an explicit check for ' '.

Part VII — Lab 6: Anagrams

Question: Check whether two strings are anagrams: same characters with the same frequencies, ignoring case and spaces.

Examples:

```
listen
silent
-> Yes

hello
world
-> No
```

The core idea: frequency array

Instead of trying to match every character with another character, count how many times each character appears.

```
int freq[26] = {0};
```

For lowercase English letters:

```
freq['a' - 'a'] -> count of a
freq['b' - 'a'] -> count of b
...
freq['z' - 'a'] -> count of z
```

For each valid character in the first string, increment its count. For the second string, decrement the same count. If all entries finish at zero, the strings are anagrams.

Why use character - 'a'?

```
char c = 'd';

int index = c - 'a';   // 3
```

Because letters are stored in increasing character-code order. This maps a–z to indices 0–25.

Complete anagram program

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main(void) {
    char s1[300], s2[300];
    int freq[26] = {0};
    int ok = 1;

    printf("Enter␣first␣string:␣");
    fgets(s1, sizeof(s1), stdin);

    printf("Enter␣second␣string:␣");
    fgets(s2, sizeof(s2), stdin);

    for (int i = 0; s1[i] != '\0'; i++) {
        if (isalpha((unsigned char)s1[i])) {
```

```
            char c = tolower((unsigned char)s1[i]);
            freq[c - 'a']++;
        }
    }

    for (int i = 0; s2[i] != '\0'; i++) {
        if (isalpha((unsigned char)s2[i])) {
            char c = tolower((unsigned char)s2[i]);
            freq[c - 'a']--;
        }
    }

    for (int i = 0; i < 26; i++) {
        if (freq[i] != 0) {
            ok = 0;
            break;
        }
    }

    if (ok)
        printf("Yes\n");
    else
        printf("No\n");

    return 0;
}
```

Dry run: listen vs silent

```
listen:
l +1
i +1
s +1
t +1
e +1
n +1

silent:
s -1
i -1
l -1
e -1
n -1
t -1
```

Every frequency becomes 0 -> Yes

What if spaces and punctuation should be ignored?

The `isalpha()` condition already ignores spaces, digits, and punctuation. If digits should count as characters too, use `isalnum()` and design the frequency array accordingly.

Part VIII — Lab 6 Optional: Compress a String

1. Logic / Approach

We need to compress a string by counting consecutive repeated characters.
Example:

```
aabccccaaa
```

Break it into groups:

```
aa      -> a2
b       -> b1
ccccc   -> c5
aaa     -> a3
```

Therefore:

```
a2b1c5a3
```

Important
We count only consecutive characters.
For:

```
aaabbbaa
```

the groups are:

```
aaa -> a3
bbb -> b3
aa  -> a2
```

So the result is:

```
a3b3a2
```

We do not combine the two groups of a.

Overall logic:

1. Take the current character.
2. Count how many times it occurs consecutively.
3. Store the character in the compressed string.
4. Store its count.
5. Continue from the next different character.
6. After compression, compare the compressed length with the original length.
7. If the compressed string is shorter, print it.
8. Otherwise, print the original string.

Example:

```
abcdef
```

would become:

```
a1b1c1d1e1f1
```

Since this is longer than abcdef, we return:

```
abcdef
```

2. C Program

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char s[500];
    char compressed[1000];

    printf("Enter␣a␣string:␣");
    fgets(s, sizeof(s), stdin);

    s[strcspn(s, "\n")] = '\0';

    int len = strlen(s);
    int pos = 0;

    for (int i = 0; i < len; )
    {
        char current = s[i];
        int count = 0;

        /* Count consecutive occurrences */
        while (i < len && s[i] == current)
        {
            count++;
            i++;
        }

        /* Store the character */
        compressed[pos++] = current;

        /* Convert count into characters */
        char temp[20];
        int t = 0;

        while (count > 0)
        {
            temp[t++] = '0' + (count % 10);
            count /= 10;
        }

        /* Store digits in correct order */
        while (t > 0)
```

```
        {
            compressed[pos++] = temp[--t];
        }
    }

    compressed[pos] = '\0';

    /* Return original if compression is not beneficial */
    if (pos < len)
        printf("Compressed string: %s\n", compressed);
    else
        printf("Compressed string: %s\n", s);

    return 0;
}
```

3. Code Explanation

Reading the string:

```
fgets(s, sizeof(s), stdin);
```

`fgets()` reads the complete line, including spaces if present.
It may also store the newline `'\n'`, so we remove it:

```
s[strcspn(s, "\n")] = '\0';
```

Finding the length:

```
int len = strlen(s);
```

`strlen()` gives the number of characters before `'\0'`.
Finding a group:

```
char current = s[i];
int count = 0;
```

current stores the character whose consecutive occurrences we are going to count.
Then:

```
while (i < len && s[i] == current)
{
    count++;
    i++;
}
```

For:

`aabccccccaaa`

initially:

`current = 'a'`

The loop counts:

`a`
`a`

so:

`count = 2`

Then i is positioned at b.
Store the character:

```
compressed[pos++] = current;
```

For the first group:

`compressed = "a"`

Convert the count into characters.
Suppose:

`count = 5`

We need to store the character '5', not the integer 5.
The expression:

```
'0' + (count % 10)
```

converts a digit into its character representation.
For example:

```
'0' + 5 -> '5'
```

The loop:

```
while (count > 0)
{
    temp[t++] = '0' + (count % 10);
    count /= 10;
}
```

extracts the digits from right to left.
For count = 12:

```
12 % 10 -> 2
12 / 10 -> 1

1 % 10 -> 1
1 / 10 -> 0
```

So temp contains:

```
2 1
```

which is backwards.
Store the digits in the correct order:

```
while (t > 0)
{
    compressed[pos++] = temp[--t];
}
```

This reverses the extracted digits.
So:

```
temp = 2 1
```

becomes:

```
compressed = 1 2
```

Therefore a count of 12 is correctly stored as:

```
"12"
```

End the compressed string:

```
compressed[pos] = '\0';
```

This is necessary because compressed is a C string and must end with the null character.
Check whether compression is useful:

```
if (pos < len)
```

Here:

```
pos = compressed string length
len = original string length
```

If:

```
compressed length < original length
```

we print the compressed string.
Otherwise, we print the original string.
For example:

```
aabccccccaaa
original length = 11
compressed length = 8
```

So:

```
a2b1c5a3
```

is printed.
But:

```
abcdef
```

has compressed form:

```
a1b1c1d1e1f1
```

which is longer, so we print:

```
abcdef
```

Part IX — Common C Concepts Behind These Labs

1. Arrays and indices

```
char s[100];

s[0]   // first character
s[1]   // second character
s[i]   // character at index i
```

Valid indices for `char s[100]` are 0 through 99. If the array is used as a string, one position must be available for `'\0'`.

2. The null character is not the same as '0'

```
'0'   // character zero
0     // integer zero
'\0'  // null character
```

The null character `'\0'` has numeric value zero, but conceptually it marks the end of a C string. Do not confuse it with the visible character `'0'`.

3. & in scanf()

```
int n;
scanf("%d", &n);
```

`scanf` needs the address of n so that it can place the input value into the variable. For a character array, `scanf("%s", s)` does not use `&s` because the array name already behaves like an address in this context.

4. while versus for

Use while when the number of iterations is controlled by a changing condition, such as repeatedly removing digits. Use for when the iteration structure is naturally count-based, such as exactly 20 Taylor terms.

5. Temporary variable for swapping

```
temp = a;
a = b;
b = temp;
```

Without temp, assigning `a = b` would destroy the original value of a. The temporary variable keeps it safe.

6. Functions make larger programs easier

For the 15-puzzle, separating `print_board()` and `move_blank()` keeps main() simple. The same principle can be used for string cleaning, rotation, compression, and frequency counting.

Lab 9 — Sensor Stabilization: Using bool Variables to Check Array Conditions

Problem

A sensor produces n readings:

```
a[0], a[1], ..., a[n-1]
```

Find the first index i such that:

```
|a[i+1] - a[i]| <= tolerance
```

If no such index exists, return -1.

Logic / Approach

Check every pair of consecutive readings from left to right.
For each index i:

```
difference = |a[i+1] - a[i]|
```

Since the values are `double`, use `fabs()` from `<math.h>`.

- If `difference <= tolerance`, return `i`.
- If no such pair is found, return `-1`.

Example

Readings:

```
10  7  5.5  5.2  5.1
```

Differences:

```
|7 - 10|    = 3
|5.5 - 7|   = 1.5
|5.2 - 5.5| = 0.3
|5.1 - 5.2| = 0.1
```

If:

```
tolerance = 0.5
```

Check from left to right:

```
i = 0 -> 3   > 0.5
i = 1 -> 1.5 > 0.5
i = 2 -> 0.3 <= 0.5
```

Therefore:

```
Answer = 2
```

C Code

```
#include <stdio.h>
#include <math.h>

int firstStable(double a[], int n, double tolerance)
{
    for(int i = 0; i < n - 1; i++)
    {
        double difference = fabs(a[i + 1] - a[i]);

        if(difference <= tolerance)
            return i;
    }

    return -1;
}

int main()
{
    int n;
    scanf("%d", &n);

    double a[1000];

    for(int i = 0; i < n; i++)
        scanf("%lf", &a[i]);

    double tolerance;
    scanf("%lf", &tolerance);

    printf("%d\\n", firstStable(a, n, tolerance));

    return 0;
}
```

Code Explanation

```
for(int i = 0; i < n - 1; i++)
```

We compare:

```
a[0] with a[1]
a[1] with a[2]
...
a[n-2] with a[n-1]
```

So the loop goes only up to `n-2`.

```
double difference = fabs(a[i + 1] - a[i]);
```

Calculates the absolute difference between two consecutive readings.

```
if(difference <= tolerance)
    return i;
```

As soon as we find the first valid pair, return its index.
If the loop finishes without finding one:

```
return -1;
```

Lab 10 — Minesweeper: Checking Conditions Efficiently, Passing and Calling 2D Arrays

Problem

We are given an $n \times m$ grid where:

```
0 -> Empty cell
1 -> Mine
```

For every cell:

- If it contains a mine, print `-1`.
- Otherwise, print the number of mines in its neighboring cells.

A cell can have up to 8 neighbors:

```
  X
```

Here, X is the current cell.

Logic / Approach

Process every cell one by one.
For each cell (i, j):
Step 1: Check whether it is a mine.

```
if(grid[i][j] == 1)
```

If it is a mine, print:

```
-1
```

Step 2: Otherwise, check all 8 neighboring cells.
We use:

```
di = -1, 0, 1
dj = -1, 0, 1
```

The neighboring position is:

```
ni = i + di
nj = j + dj
```

This generates all positions around (i, j).
We skip:

```
di = 0 and dj = 0
```

because that is the current cell itself.
Step 3: Make sure the neighbor is inside the grid:

```
ni >= 0 && ni < n &&
nj >= 0 && nj < m
```

Step 4: If the neighboring cell contains `1`, increase the count.

Example

Given:

```
0 1 0
0 0 0
1 0 0
```

For the center cell:

```
0 1 0
0 X 0
1 0 0
```

Its neighboring mines are:

```
    1

1
```

So:

```
Answer = 2
```

C Code

```
#include <stdio.h>

int main()
{
    int n, m;
    scanf("%d␣%d", &n, &m);

    int grid[100][100];

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
            scanf("%d", &grid[i][j]);
    }

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            if(grid[i][j] == 1)
            {
                printf("-1");
            }
            else
            {
                int count = 0;

                for(int di = -1; di <= 1; di++)
                {
                    for(int dj = -1; dj <= 1; dj++)
                    {
                        if(di == 0 && dj == 0)
                            continue;

                        int ni = i + di;
                        int nj = j + dj;

                        if(ni >= 0 && ni < n &&
                           nj >= 0 && nj < m)
                        {
                            if(grid[ni][nj] == 1)
                                count++;
                        }
                    }
                }

                printf("%d", count);
            }
```

```
            if(j != m - 1)
                printf("␣");
        }

        printf("\\n");
    }

    return 0;
}
```

Code Explanation

The two outer loops process every cell:

```
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
```

So (i, j) represents the current cell.

For a mine:

```
if(grid[i][j] == 1)
{
    printf("-1");
}
```

Otherwise:

```
int count = 0;
```

We start with zero neighboring mines.

Then:

```
for(int di = -1; di <= 1; di++)
{
    for(int dj = -1; dj <= 1; dj++)
```

This checks all possible combinations:

```
(-1,-1)  (-1,0)  (-1,1)
( 0,-1)  ( 0,0)  ( 0,1)
( 1,-1)  ( 1,0)  ( 1,1)
```

We skip:

```
if(di == 0 && dj == 0)
    continue;
```

because $(0, 0)$ means the current cell.

Then calculate the neighbor:

```
int ni = i + di;
int nj = j + dj;
```

Before accessing the grid, check the boundaries:

```
if(ni >= 0 && ni < n &&
nj >= 0 && nj < m)
```

If that neighboring cell is a mine:

```
if(grid[ni][nj] == 1)
    count++;
```

Finally, print the number of neighboring mines:

```
printf("%d", count);
```

Part XI — Quick Logic Templates

Template A: Process every character in a string

```
for (int i = 0; s[i] != '\0'; i++) {
    char c = s[i];

    // process c
}
```

Template B: Process only letters, ignoring case

```
for (int i = 0; s[i] != '\0'; i++) {
    if (isalpha((unsigned char)s[i])) {
        char c = tolower((unsigned char)s[i]);
        // process c
    }
}
```

Template C: Extract digits from right to left

```
while (n > 0) {
    int digit = n % 10;
    n = n / 10;
}
```

Template D: Two-pointer string scan

```
int left = 0;
int right = strlen(s) - 1;

while (left < right) {
    // move left to a valid character
    // move right to a valid character
    // compare
    left++;
    right--;
}
```

Template E: Frequency array

```
int freq[26] = {0};

for (int i = 0; s[i] != '\0'; i++) {
    if (isalpha((unsigned char)s[i])) {
        char c = tolower((unsigned char)s[i]);
        freq[c - 'a']++;
    }
}
```

FINAL TIPS

Sensor Stabilization

Check every consecutive pair:

`|a[i+1] - a[i]| <= tolerance`

Return the **first** valid index.

2D Minesweeper

For every cell, check its 8 neighboring positions using:

```
di = -1, 0, 1
dj = -1, 0, 1
```

Calculate:

```
ni = i + di
nj = j + dj
```

Check boundaries, count the mines, and print the count.